

哈尔滨工业大学

编译系统实验报告（一）

姓名	李凡丁
学号	7203610625
学院	计算机科学与技术
教师	朱庆福

2023 年 4 月

一、实验目的

1. 巩固对词法分析与语法分析的基本功能和原理的认识
2. 能够应用自动机的知识进行词法分析和语法分析
3. 理解并处理词法分析与语法分析中的异常和错误

二、实验环境

1. Ubuntu 20.04
2. GCC 9.4.0
3. Flex 2.6.4
4. Bison 3.5.1

三、实验过程

1. 本次实验用到的数据结构

本次实验我们通过语法分析树进行语法分析，所以我们用到的就是一个树结构。为了节省空间，我们采用多叉树的二叉树表示法（即左子节点是子节点，右子节点是兄弟节点）的表示方法，从而可以使用链表进行树的存储而不占用多余的存储空间也可以确切知道树的大小。每个树节点就是我们语法分析中的一个语法单元，我们采用一个联合体（union）进行树节点的信息的表示，同时我们再进行树节点的基本信息的表示就完成了数据结构的设计。

由于我们使用链表来最大程度上减少无效空间的使用，这样我们对于特定语法单元的子节点数量就不能够进行事先的预知，这样我们就要用到 `stdarg` 库中的不定参数数量的 `va_list` 数据结构来进行子节点的存储和相关操作。

在最终检验无误后，我们需要对于整段代码进行输出。输出的过程本质上就是对我们的语法分析树进行遍历，所以我们还需在我们的树结构的基础上实现树的深度优先遍历。这样我们就完成了数据结构的设计。

```
struct Parsetree
{
    int line;
    char *Token;
    int isleaf;
    union
    {
        char* Id_Type;
        int intval;
        float floatval;
    };
    struct Parsetree *firstchild,*nxtbro;
};

static void dfs(Treenode now, int depth)
{
    if (now==NULL||now->Token==NULL) return;
    printf("%s",2*depth,"");
    printf("%s",now->Token);
    if((!strcmp(now->Token,"ID"))||(!strcmp(now->Token,"TYPE")))
        printf(" : %s",now->Id_Type);
    else if(!strcmp(now->Token,"INT"))
        printf(" : %d",now->intval);
    else if(!strcmp(now->Token,"FLOAT"))
        printf(" : %f",now->floatval);
    else if(!now->isleaf)
        printf(" (");
    puts("");
    dfs(now->firstchild,depth+1);
    dfs(now->nxtbro,depth);
}
```

图 1：实验用到的数据结构的主要代码

2. 基于 flex 的词法分析过程

我们通过 lexical.l 文件的编写实现对于词法的分析，在这个过程中比较重要的一步就是书写特定词的正则表达式。较为复杂的三个部分分别是 INT, FLOAT 和 ID。对于其它的部分，我们只需要将名称与对应的符号进行匹配即可。在最终的词法分析中，我们将 C++ 程序拆分成一个一个的词进行输出。

```
INT [1-9][0-9]*|0          LC "{  
                             RC "}"  
FLOAT {INT}\.[0-9]+        LB "[  
                             RB "]"  
ID [a-zA-Z_][a-zA-Z_0-9]*  LP "("  
                             RP ")"
```

图 2: Flex 代码的编写 (部分)

我们同样也会针对一些特定的错误词法设计出相对应的错误解决办法，在本实验中我们将这种错误记为了 A 类错误。

```
{INT}+{ID} {hasFault = 1; printf("Error type A at Line %d: Illegal ID '%s'\n", yylineno, yytext); }  
\.{INT} {hasFault = 1; printf("Error type A at Line %d: Illegal float number '%s'\n", yylineno, yytext); }  
. {hasFault = 1; printf("Error type A at Line %d: Mysterious characters '%s'\n", yylineno, yytext); }  
%%
```

图 3: 错误的处理形式

基于我们编写的 lexical.l 以及我们稍后编写的主文件 main.c 即可完成词法分析这一部分。

3. 基于 bison 的语法分析过程

我们通过 syntax.y 文件的编写实现对于语法的分析，在这个过程中比较重要的一步就是错误的匹配。我们会将各种正确情况进行枚举，然后在 syntax.y 代码中列举正确的情况的后续操作，而不符合正确情况的我们就统一使用 error 单元进行匹配。当然在做这些之前我们先要规定整个语法分析过程中的终结符与非终结符，因为我们的语法分析是基于自动机的知识进行实现的。

%type <node> Program ExtDefList ExtDef ExtDeclList	// High-level Definitions	%token <node> INT FLOAT ID
%type <node> Specifier StructSpecifier OptTag Tag	// Specifiers	%token <node> SEMI COMMA
%type <node> VarDec FunDec VarList ParamDec	// Declarators	%token <node> ASSIGNOP RELOP
%type <node> CompSt StmtList Stmt	// Statements	%token <node> PLUS MINUS STAR DIV
%type <node> DefList Def Dec Declist	// Local Definitions	%token <node> AND OR DOT NOT
%type <node> Exp Args	// Expressions	%token <node> LP RP LB RB LC RC
		%token <node> TYPE STRUCT RETURN
		%token <node> IF ELSE WHILE

图 4: 我们定义的终结符和非终结符

```
ExtDef : Specifier ExtDeclList SEMI { $$ = newnode(@$.first_line, "ExtDef", 3, $1, $2, $3); }  
      | Specifier SEMI { $$ = newnode(@$.first_line, "ExtDef", 2, $1, $2); }  
      | Specifier FunDec CompSt { $$ = newnode(@$.first_line, "ExtDef", 3, $1, $2, $3); }  
      | error SEMI { hasFault = 1; }  
      ;
```

图 5: syntax.y 主代码 (部分)

我们每当处理到一个合法的节点，首先我们需要为其声明一个树节点，而后我们需要将这个节点的信息以及这个节点的子节点的数量以及编号全部传入声明节点的函数。我们判断终结符的方式是判断当前节点的子节点数量是否等于 0，若为 0 则是，反之则不是。对于终结符我们需要提取其信息，对于非终结符则无需提取信息，只需向下继续处理子节点即可。处理节点这部分代码体现在 treenode.h 中如下：

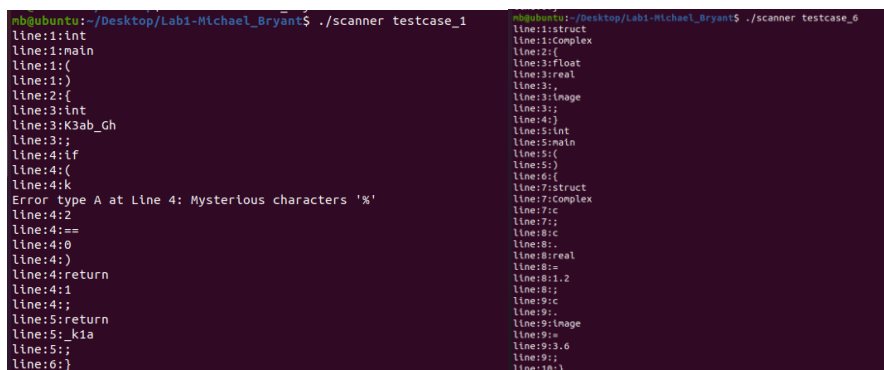
```
if (amount==0)  
{  
    rt->isleaf=1;  
    rt->firstchild=NULL;  
    if(!strcmp(TOKEN,"ID")||!strcmp(TOKEN,"TYPE"))  
    {  
        char *str;  
        str=(char*)malloc(sizeof(char)*40);  
        strcpy(str,yytext);  
        rt->id_type=str;  
    }  
    else if(strcmp(TOKEN,"INT"))  
        rt->intval=atoi(yytext);  
    else if(strcmp(TOKEN,"FLOAT"))  
        rt->floatval=atof(yytext);  
}  
else  
{  
    va_list list;  
    va_start(list,amount);  
    ch=va_arg(list,treenode);  
    rt->firstchild=ch;  
    if (amount==1)  
    {  
        for (int i=1;i<=amount-1;i++)  
        {  
            ch->nextbro=va_arg(list,treenode);  
            ch->nextbro->nextbro=NULL;  
        }  
    }  
    va_end(list);  
    return rt;  
}
```

图 6: 处理节点的代码

4. 实验的操作流程

- (1) 编写 lexical.l 代码, 利用 flex 进行编译得到 lex.yy.c
- (2) 编写主函数 main.c 与 lex.yy.c 共同编译得到词法分析工具 scanner
- (3) 编写 syntax.y 代码, 在其中引用 lex.yy.c, 利用 bison 进行编译得到 syntax.tab.c 和 syntax.tab.h
- (4) 在 lexical.l 中引用 syntax.tab.h 重新编译得到新的 lex.yy.c
- (5) 编写主函数 main.c, 与 syntax.tab.c 共同编译得到语法分析工具 parser

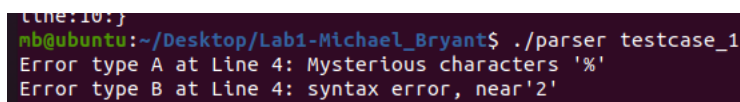
5. 实验的运行结果 (仅展示部分)



```
mb@ubuntu:~/Desktop/Lab1-Michael_Bryant$ ./scanner testcase_1
line:1:int
line:1:main
line:1:(
line:1:
line:2:{
line:3:int
line:3:K3ab_Gh
line:3;;
line:4:if
line:4:(
line:4:k
line:4:
Error type A at Line 4: Mysterious characters '%'
line:4:2
line:4:==
line:4:0
line:4:)
line:4:return
line:4:1
line:4;;
line:5:return
line:5:k1a
line:5;;
line:6:}
```

```
mb@ubuntu:~/Desktop/Lab1-Michael_Bryant$ ./scanner testcase_6
line:1:struct
line:1:Complex
line:2:{
line:3:float
line:3:real
line:3;;
line:3:image
line:3;;
line:4;}
line:5:int
line:5:main
line:5:(
line:5:{
line:6:struct
line:6:Complex
line:6:c
line:6;;
line:6:real
line:6:=
line:6:1.2
line:6;;
line:6:c
line:6;;
line:6:real
line:6:=
line:6:3.6
line:6;;
line:6:}
```

图 7: scanner 运行结果 (左为有错误 右为无错误)



```
line:10:}
mb@ubuntu:~/Desktop/Lab1-Michael_Bryant$ ./parser testcase_1
Error type A at Line 4: Mysterious characters '%'
Error type B at Line 4: syntax error, near '2'
```

图 8: parser 分析出语法错误的情况



```
mb@ubuntu:~/Desktop/Lab1-Michael_Bryant$ ./parser testcase_7
Program (1)
ExtDefList (1)
  Specifier (1)
    StructSpecifier (1)
      Tag (1)
        ID: Complex
      Declist (1)
        VarDec (1)
          ID: c
          SEMI
          StmtList (9)
            Stmt (2)
              Exp (9)
                Exp (9)
                  ID: c
                  DOT
                  ID: real
                  ASSIGNOP
                  Exp (9)
                    FLOAT: 1.200000
            SEMI
            StmtList (10)
              Stmt (10)
                Exp (10)
                  Exp (10)
                    ID: c
                    DOT
                    ID: image
                    ASSIGNOP
                    Exp (10)
                      FLOAT: 3.000000
            SEMI
          RC
        ExtDefList (6)
          Specifier (6)
            Type: int
          FunDec (6)
            LP
            RP
            Compst (7)
              LC
              DefList (6)
                Def (6)
```

图 9: parser 分析正确代码段的情况

四、实验心得与体会

1. 通过本次实验, 我学会了借助 flex, bison 等工具完成对于词法和语法正确性的分析
2. 在实验的过程中, 我巩固了自动机这门课的相关知识, 并将其应用至本次实验中, 利用其对于结果做出了正确的分析。
3. 在实验中, 我也巩固了对于正则表达式的理解, 对于 C++ 语言中给定的词法和文法利用正则表达式做出了正确的表示。